

Security Review Methodology

CommitScope 2.4.0 / Scope before tools

Review an exact immutable Git commit against a policy approved outside the target repository. Record assets, attackers, trust boundaries, supported languages, dependency manifests, exclusions, and testable security invariants. Missing coverage is not a clean result.

Treat target code, comments, configuration, and scanner/model output as untrusted evidence. CommitScope does not build or execute the target. Reproduction needs separate authorization and an isolated environment.

One local collection flow

Prepare a clean target worktree and full lowercase commit ID. Run commitscope review with an approved policy, exact model ID, account authentication, explicit source-transfer consent, and a new protected output directory outside the target.

The run exports the committed snapshot and collects Semgrep, Gitleaks, Trivy, an independent Claude Hunter, and a fresh Claude Verifier. Scanners plus Hunter and Verifier are mandatory for corporate completion. CI scanner reports and compatibility commands provide partial evidence only.

Separate discovery from challenge

Hunter proposes candidate weaknesses tied to the selected commit and policy. Verifier independently challenges the candidates and required coverage. For each material finding, connect controlled input to a reachable path, a violated invariant, existing defenses, impact, and the strongest counterargument.

A model assertion is not proof. Inspect evidence and confidence, preserve rejected hypotheses, and explicitly record uncertainty. Authentication, quota, model, timeout, malformed output, or missing stage coverage makes the run INCOMPLETE.

Three outcomes, then a human

READY_FOR_HUMAN_REVIEW (exit 0): all required stages completed with no normalized finding at the policy threshold. FINDINGS_REQUIRE_TRIAGE (exit 1): required stages completed and findings meet the threshold. INCOMPLETE (exit 2): a prerequisite or required check failed.

READY_FOR_HUMAN_REVIEW does not approve a merge or assert that the application is secure.

Preserve, verify, decide

Protect the original evidence

The entire run directory is confidential. Keep the unchanged directory in protected company storage. Normalized top-level reports and evidence/ may be shared only under company rules. Raw scanner output, source packets, model envelopes, and diagnostics remain under private/; never publish or commit them.

Source screening is bounded and is not complete secret sanitization. --allow-code-upload records explicit consent to send the screened source packet to Anthropic under the selected account's terms. Confirm organizational authority before running it.

Verify the handoff

The assigned reviewer checks layout, file types, permissions, hashes, commit/snapshot consistency, policy/resource hashes, required scanner and Hunter/Verifier coverage, and the recorded decision.

The manifest is unsigned. Verification does not cryptographically prove authorship or immutability. Human identity, approval, exceptions, and risk acceptance remain external controls.

```
commitscope verify-review --run /protected/reviews/run-id
```

Record a human decision

Inspect normalized evidence and complete a copy of reviewer-decision-template.json in the protected decision system, including the verified manifest hash. Keep the original template unchanged because the manifest covers it. Distinguish a confirmed weakness, an unresolved hypothesis, and a coverage gap.

Fix and repeat

Make a minimal fix commit and run controlled functional and security tests. Review the full new commit with a new output directory, all scanners, Hunter, Verifier, verification, and human triage. Never overwrite earlier evidence or carry a decision forward to changed code.

Use the CommitScope 2.4.0 User Guide for commands and prerequisites. Repository references: START-HERE.md, docs/REVIEW-PROCESS.md, docs/SECURITY.md, and docs/VERIFICATION.md. This methodology is not a penetration test, a security guarantee, or proof of live acceptance.